Celebal Technologies — Summer Internship 2026

Week 8 Assignment

1. Project Overview

The Week 8 assignment was developed as a complete **local data analytics pipeline for an e-commerce business** using Python, Pandas, SQLite, and SQL.

The purpose of the project was to work with synthetic but realistic e-commerce data containing deliberately introduced data-quality problems. The raw datasets were processed through multiple stages including **data generation, quality validation, cleaning, relational database loading, analytical querying, reporting, and automated testing**.

The implementation demonstrates how raw transactional data can be transformed into structured and reliable information that can subsequently be used for business analysis.

2. What the System Does

The project follows an end-to-end data processing workflow:

Generate → Validate → Clean → Store → Analyze → Report → Test

The system performs the following major activities:

- Generates four related e-commerce datasets.
- Introduces controlled data-quality problems.
- Detects and resolves applicable data issues.
- Checks relationships between different datasets.
- Stores the cleaned records in SQLite.
- Executes 17 analytical SQL queries.
- Generates reports through a command-line interface.
- Tests important edge cases using Python unittest.

The entire implementation runs locally and does not require any web server, API, cloud service, or external application.

3. Technology Stack

Technology	Role in the Project
Python	Core programming and data-processing logic
Pandas	CSV processing and data cleaning
SQLite	Relational database storage
SQL	Business and analytical queries
unittest	Automated validation and edge-case testing

4. Data Model

The system works with four interconnected datasets.

Dataset	Records	Description
customers.csv	1,000	Customer information, contact details, registration data, and customer type
products.csv	500	Product catalogue, categories, subcategories, and pricing
orders.csv	1,000	Order information including dates, customers, status, and region
order_items.csv	2,000	Individual products purchased, quantities, prices, and discounts

A fixed random seed of **42** is used during generation so that the raw data can be reproduced consistently.

5. Data Quality Simulation

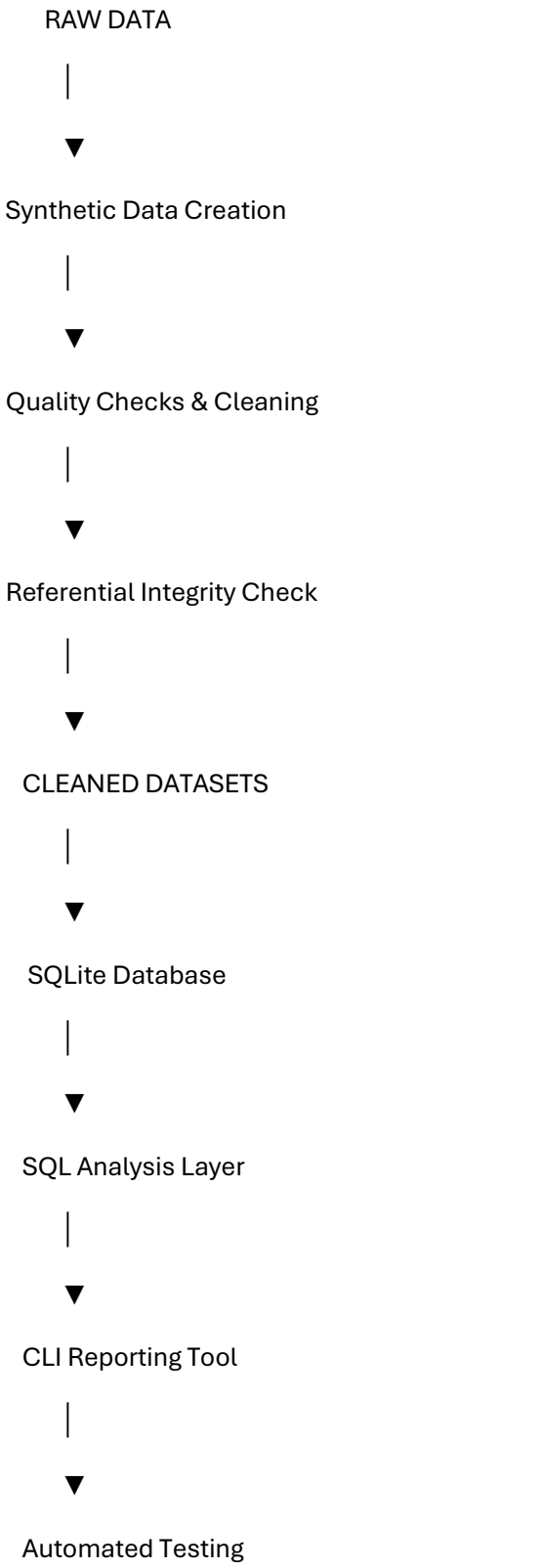
To make the project closer to a real-world data engineering scenario, several intentional issues were introduced into the raw datasets.

Problem Introduced	**Records Affected**
Missing customer_id	50 orders
Negative quantities	60 order items
Incorrect date format	50 orders
Product-name formatting defects	30 products
Invalid email addresses	20 customers
Orphan order items	30 items

The negative quantities represent returned products, while missing customer IDs can represent legitimate guest checkouts.

6. Data Processing Pipeline

The project processes the raw data in several stages.



This separation keeps data generation, cleaning, database loading, analytics, reporting, and testing independent from each other.

7. Cleaning and Validation Process

The raw CSV files are processed using Python and Pandas before being inserted into the database.

The main processing functions include:

clean_orders()

Responsible for correcting malformed dates while retaining valid guest checkout records where customer_id is missing.

clean_products()

Removes unnecessary whitespace and standardizes product names.

validate_emails()

Checks customer records and identifies invalid email addresses.

check_referential_integrity()

Verifies whether order items refer to valid orders.

Cleaning Summary

Processed Output	**Final Result**
Customers cleaned	1,000
Products cleaned	500
Orders cleaned	1,000
Valid order items	1,970
Orphan items separated	30
Dates corrected	50
Invalid emails identified	20
Negative return quantities retained	60

The orphan records are preserved separately instead of being permanently deleted:

orphan_order_items.csv

A detailed quality report is also produced: data/cleaned/data_quality_report.md

8. Database Implementation

After validation, the cleaned datasets are loaded into a SQLite database named:

ecommerce.db

The database contains four relational tables.

Table	Rows
customers	1,000
products	500
orders	1,000
order_items	1,970

Database Integrity Checks

The database was validated using SQLite integrity commands.

Results:

- PRAGMA integrity_check → **OK**
- PRAGMA foreign_key_check → **0 violations**
- Valid order items reference existing orders and products.
- Guest orders with NULL customer_id remain valid.

9. Analytical SQL Layer

A total of **17 SQL analysis tasks** were implemented as part of the project.

#	Analysis Performed	Main SQL Concept
1	Revenue by Category	Aggregation
2	Top 10 Customers	Aggregation + Ordering
3	Previous 12 Months	Date Filtering
4	Undelivered Customers	Conditional Aggregation
5	Excessive Returns	GROUP BY + HAVING
6	Category Return Rate	Conditional Aggregation
7	Running Revenue Totals	Window SUM()
8	Product Dense Ranking	DENSE_RANK()
9	Customer LAG / LEAD	Window Functions
10	Multi-Level Customer Segmentation	CTE
11	Customer Value Quartiles	NTILE(4)
12	Year-over-Year Analysis	Date-Based Comparison
13	First / Last Purchase Values	FIRST_VALUE() / LAST_VALUE()
14	Cumulative Revenue Distribution	Window Analysis
15	Customer Cohort Analysis	Cohort / Retention
16	Consecutive Order Analysis	Self Join + Window
17	Frequently Purchased Together	Self Join

All **17 queries were successfully executed** against the final SQLite database.

10. Advanced SQL Concepts Implemented

Running Revenue Totals

Daily revenue is calculated by region and then accumulated chronologically using a window function:

```
SUM(daily_revenue)
OVER (
  PARTITION BY region_code
  ORDER BY order_date
)
```

Product Ranking

Products are ranked within their respective categories according to revenue.

DENSE_RANK() is used so products having equal revenue receive the same rank without unnecessary gaps.

Customer Order Gap Analysis

LAG() is used to compare a customer's current order with the previous order.

The difference between consecutive orders is calculated in days. Customers whose average gap exceeds 30 days are classified as **At Risk**.

CTE-Based Segmentation

Customer revenue is first calculated at the monthly level and then passed into subsequent CTEs to classify customers into:

- High
- Medium
- Low

segments according to the assignment criteria.

NTILE Customer Segmentation

Customers with purchasing activity are divided into four lifetime-value groups using:

NTILE(4)

The resulting groups are:

Platinum → Gold → Silver → Bronze

Cohort Analysis

Customers are grouped according to their registration month. Their subsequent purchasing activity is tracked across month 0 to month 3 to measure retention.

Frequently Bought Together

A self-join is used to identify products purchased within the same order.

The implementation avoids:

- Same-product pairs
- Reversed duplicate pairs

by enforcing an ordered product-pair condition.

11. Command-Line Reporting

The project also includes a Python-based command-line reporting utility connected directly to SQLite.

The reporting tool supports:

Capability	Supported
Daily reports	✅
Weekly reports	✅
Monthly reports	✅
Custom date range	✅
Total orders	✅
Revenue	✅
Unique customers	✅
Top 3 products	✅
Previous-period comparison	✅

Example command:

python src/cli/report.py --type Monthly --start 2023-01-01 --end 2023-01-31

The reporting tool was also tested with an empty date range and handled the absence of sales without crashing.

12. Handling Exceptional Scenarios

The implementation was tested against the important edge cases specified in the assignment.

Scenario	Handling
Orphan order item	Identified and isolated
Discount greater than 100%	Safely handled and validated
Zero quantity	Processed without failure and contributes zero revenue
Future order date	Detected successfully

All four edge-case tests passed successfully using Python's unittest framework.

13. Final Testing Status

The complete project was verified after implementation.

Module / Feature	Result
Synthetic data generation	**PASS**
Data cleaning	**PASS**
Data validation	**PASS**
Raw data preservation	**PASS**
SQLite loading	**PASS**
Database integrity	**PASS**
Foreign-key validation	**PASS — 0 violations**
SQL analysis	**PASS — 17/17**
CLI reporting	**PASS**
Edge-case tests	**PASS — 4/4**

14. Project Deliverables

The final implementation contains:

- Four raw CSV datasets
- Cleaned datasets
- Orphan order-item dataset
- Data-quality report
- SQLite database
- 17 SQL analysis scripts
- Python data-generation module
- Python data-cleaning module
- Database-loading module
- Analytics execution module
- CLI reporting tool
- Automated test suite
- Project documentation

15. Conclusion

The Week 8 assignment successfully demonstrates an **end-to-end e-commerce data analytics workflow** using Python, Pandas, SQLite, and SQL.

Starting with synthetic raw data containing controlled quality issues, the project performs data validation and cleaning, maintains referential integrity, stores the processed information in a relational database, and applies a wide range of SQL techniques for business analysis.

The project also goes beyond basic querying by implementing **window functions, CTEs, customer segmentation, cohort analysis, year-over-year comparisons, product-pair analysis, and command-line reporting**. Automated tests were used to verify the required edge cases and overall system behaviour.

Overall, the implementation provided practical experience in **data cleaning, relational database management, analytical SQL, advanced query techniques, Python–SQL integration, reporting, and data-quality validation**, which are important components of a real-world Data Engineering workflow.